Metodologia de Sistemas II

Trabajo Practico Integrador (TPI)

Evolución profesional de un sistema existente: calidad, deuda técnica, refactoring y revisión técnica

Este trabajo practico integrador propone que el estudiante actúe sobre un sistema ya definido o un prototipo mínimo, no para agregar funcionalidad por acumulación, sino para mejorar su calidad interna, reducir deuda técnica y justificar decisiones de diseño profesional. **El foco de evaluación esta puesto en la capacidad de diagnosticar, refactorizar, revisar y comunicar decisiones técnicas con evidencia.**

Aspecto	Definition
Modalidad	Grupal hasta 4 integrantes. Se admite entrega individual con justificación.
Producto central	Informe PDF único, repositorio GitHub, anexo de evidencias y video de defensa/demostración.
Caso de trabajo	Sistema o modulo simple existente/prototipado: gestión de solicitudes, incidencias, turnos, pedidos, reservas u otro caso aprobado por la cátedra.
Condición clave	Debe existir comparación antes/después: versión base con problemas detectables y versión mejorada mediante refactoring seguro.

1. Objetivo del trabajo

Evaluar la aplicación integrada de los contenidos de Metodología de Sistemas II mediante el análisis y mejora de un sistema de software. El trabajo debe evidenciar la diferencia entre calidad externa e interna, identificar deuda técnica, reconocer code smells, aplicar principios SOLID y Clean Code, ejecutar refactorings seguros y realizar una revisión técnica profesional del código producido.

2. Enfoque general

- No se busca construir un sistema grande, sino analizar y mejorar profesionalmente un modulo acotado.
- La funcionalidad visible puede ser simple; la complejidad esperada esta en el diagnostico, las decisiones de diseño y la evidencia técnica.
- El equipo debe mostrar una versión inicial y una versión mejorada, justificando que el comportamiento externo se mantiene o mejora sin introducir regresiones.
- La cátedra puede solicitar defensa individual para verificar autoría, comprensión y participación real.

3. Caso de trabajo sugerido

Caso sugerido: Sistema de Gestión de Solicitudes Académicas. El sistema permite registrar solicitudes de estudiantes, clasificarlas por tipo, prioridad y estado, asignar un responsable, consultar solicitudes y generar un reporte simple.

- Entidades mínimas sugeridas: Solicitud, Estudiante/Usuario, Responsable/Area y Estado.
- Casos de uso mínimos: registrar solicitud, cambiar estado, asignar responsable, buscar/filtrar solicitudes y listar reporte básico.
- El equipo puede usar otro dominio equivalente, siempre que permita aplicar diagnostico de calidad, refactoring y code review.
- Se puede tomar un proyecto previo, un modulo desarrollado en otra materia o un prototipo construido por el equipo para este trabajo.

4. Requerimientos mínimos del sistema base

- Debe contar con al menos 3 entidades o estructuras principales.
- Debe implementar al menos 4 operaciones funcionales verificables.
- Debe persistir o simular datos mediante archivo, base de datos, memoria estructurada o repositorio equivalente.
- Debe tener una versión inicial con problemas reales de diseño o implementación que puedan ser diagnosticados.
- Debe permitir demostrar que la versión final conserva el comportamiento externo esperado.

5. Etapas obligatorias

Etapa	Título	Producción esperada
1	**Selección y descripción del sistema base**	Describir el dominio, alcance, actores, funcionalidades, tecnología utilizada y versión inicial del sistema. Incluir enlace al repositorio y evidencia de

Etapa	Título	Producción esperada
		funcionamiento.
2	**Calidad externa vs. calidad interna**	Explicar que funciona correctamente desde la mirada del usuario y que problemas existen en la estructura interna. Relacionar funcionalidad visible, mantenibilidad, legibilidad y evolución futura.
3	**Deuda tecnica y code smells**	Identificar al menos 8 code smells o debilidades técnicas. Para cada uno: evidencia, ubicación, impacto, riesgo de mantenimiento y prioridad de corrección.
4	**Aplicacion de SOLID y Clean Code**	Proponer y aplicar mejoras vinculadas a SRP, OCP, LSP, ISP y DIP cuando corresponda. Mejorar nombres, funciones, manejo de errores, cohesión, acoplamiento y límites entre componentes.
5	**Refactoring seguro**	Aplicar al menos 4 refactorings documentados. Deben incluir, como mínimo, Extract Method, Move Method o reubicación de responsabilidad, e Introduce Parameter Object o simplificación equivalente de parámetros.
6	**Pruebas y verificación del comportamiento**	Definir casos de prueba o evidencias de ejecución antes/despues. Explicar como se verifico que el refactoring no altero negativamente el comportamiento externo.
7	**Code review profesional**	Elaborar un checklist técnico y aplicarlo sobre el repositorio. Registrar observaciones, feedback profesional, correcciones realizadas y criterios de aceptación técnica.
8	**Conclusiones y aprendizaje**	Reflexionar sobre decisiones tomadas, dificultades, limites del trabajo, mejoras pendientes y relación con los contenidos de la asignatura.

6. Entregables obligatorios

6.1 Informe PDF único:

Debe incluir los siguientes apartados y contenidos:

- Portada con materia, comisión, titulo del trabajo, integrantes y fecha.
- Resumen ejecutivo de 5 a 7 líneas.
- Descripción del sistema base y alcance.
- Desarrollo de cada etapa obligatoria, respaldado con capturas, tablas, fragmentos de código para el diagnóstico y las mejoras, junto con el registro ordenado y detallado de los prompts utilizados durante las actividades (incluyendo interacciones con herramientas de IA) y sus evidencias complementarias (diagramas, informes de herramientas analíticas, archivos de prueba y documentación adicional).
- Enlace al repositorio GitHub asociado, el cual debe cumplir con la siguiente estructura:
    - Proyecto completo con los tags obligatorios de estado general: estado-inicial (para la entrega del sistema base) y estadorefactorizado (para el estado final del código).

  - Tags específicos en los commits clave para identificar el hito de cada etapa: etapa-diagnostico, etapa-refactoring, etapa-pruebas y etapa-docs.
  - README.md con descripción, instalación/uso, funcionalidades y tecnologías.
  - Carpeta /docs o equivalente con checklist de code review, evidencias y decisiones técnicas.
  - Historial de commits claro y ordenado.
- Enlace al video explicativo e identificación clara de la participación y contribución de cada integrante.
- Conclusiones grupales e individuales breves.

6.2 Video de defensa y demostración

- Duration sugerida: 10 a 15 minutos.
- Todos los integrantes deben presentarse con cámara encendida, mostrando rostro y nombre completo.
- Debe incluir problema, sistema base, diagnostico, principales code smells, refactorings aplicados, pruebas/evidencias, code review y conclusiones.
- La demostración puede realizarse con pantalla compartida del sistema, repositorio, commits o comparación antes/después.
- No se admiten voces generadas por IA. La defensa debe realizarse con voz propia.

7. Uso de inteligencia artificial

- El uso de IA está permitido como apoyo para comprender conceptos, revisar alternativas o mejorar redacción.
- Debe declararse en el informe indicando herramienta utilizada, finalidad y grado de intervención.
- Debe adjuntarse evidencia del intercambio relevante en PDF o anexo.
- La IA no reemplaza la autoría ni la defensa. El equipo debe comprender y justificar todo lo entregado.

8. Criterios de evaluacion

Criterio	Ponderación	Descripción
Diagnostico de calidad interna y externa	15%	Diferencia calidad visible y estructura interna; analiza mantenibilidad y evolución futura con ejemplos concretos.
Deuda técnica y code smells	20%	Identifica smells relevantes, evidencia ubicación, explica impacto y prioriza mejoras.
Aplicación de SOLID y Clean Code	20%	Mejora responsabilidades, nombres, funciones, errores, cohesión, acoplamiento y limites entre componentes.
Refactoring seguro	20%	Aplica tecnicas concretas y documenta antes/despues sin alterar negativamente el comportamiento externo.
Pruebas y verificación	10%	Define casos de prueba, capturas o evidencias que respalden la estabilidad funcional.

Criterio	Ponderación	Descripción
Code review profesional	10%	Usa checklist, criterios técnicos y feedback objetivo con acciones correctivas.
Presentación, repositorio y defensa	5%	Entrega completa, clara, trazable y con participación equilibrada.

9. Proceso de evaluación

- Primera instancia - Presentation
  - Entrega de PDF, repositorio, ZIP y video dentro del plazo definido por la cátedra.
  - Resultado posible: Aprobado o Requiere correcciones.
- Segunda instancia - Recuperatorio
  - Entrega de nueva versión corregida segun devolución.
  - Resultado posible: Aprobado o No aprobado.
- Instancia previa al examen final
  - Quienes no aprueben en segunda instancia deberán presentar versión actualizada una semana antes del final.
  - La cátedra confirmara si el estudiante queda habilitado para rendir.

10. Condiciones importantes

- Se considera entregado cuando están cargados todos los recursos requeridos: PDF, repositorio, ZIP y video.
- No usar datos personales reales sin consentimiento. Utilizar datos sintéticos o anonimizados.
- La entrega grupal exige participación demostrable de todos los integrantes.
- La cátedra puede realizar preguntas individuales sobre cualquier parte del trabajo.
- El trabajo no se aprueba por cantidad de código, sino por calidad del analisis, justificación, trazabilidad y mejora técnica demostrada.
- Los enlaces deben estar accesibles al momento de la corrección.

11. Estructura sugerida del informe

1. Portada.
2. Resumen ejecutivo.
3. Descripción del sistema base.
4. Calidad externa e interna.
5. Diagnostico de deuda técnica y code smells.
6. Aplicación de SOLID y Clean Code.
7. Refactorings aplicados con evidencia antes/después.
8. Pruebas o verificaciones realizadas.
9. Code review y checklist tecnico.
10. Conclusiones y mejoras futuras.
11. Anexos y enlaces.

12. Cierre conceptual

Este TPI busca que el estudiante comprenda que un sistema profesional no solo debe funcionar, sino también poder mantenerse, revisarse, extenderse y evolucionar de manera segura. La calidad interna no siempre se ve desde la interfaz, pero condiciona fuertemente el costo del cambio y la vida útil del software.